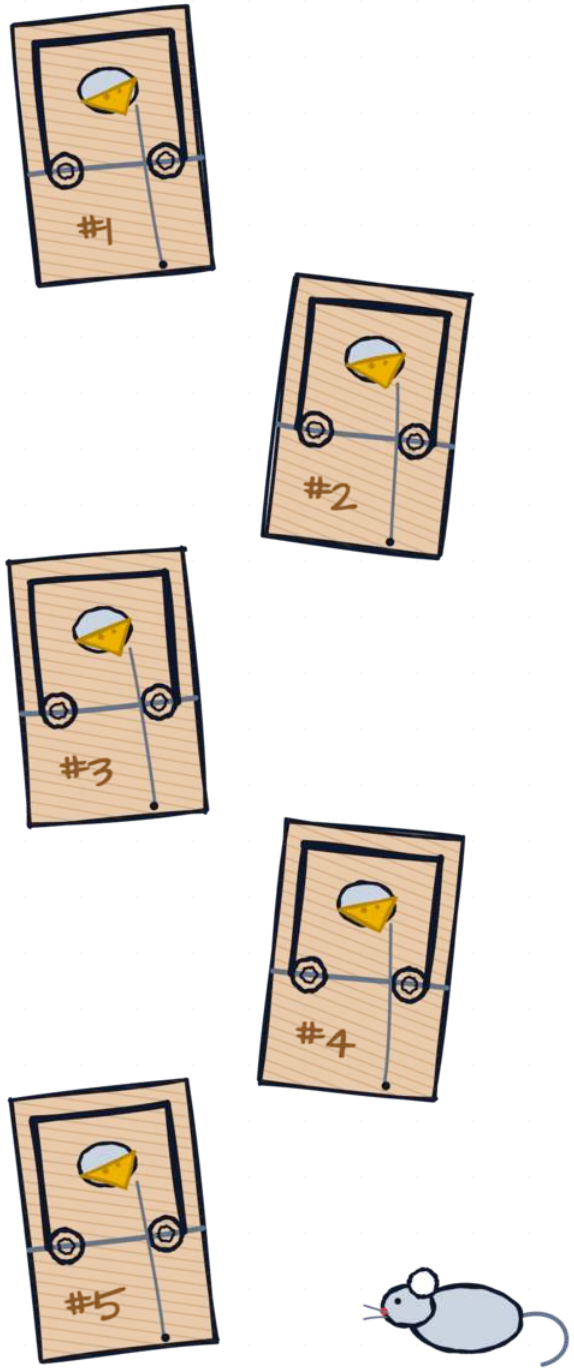


Top 5 Spring Boot Testing Mistakes Developers Trap Into

Your tests are green. But do you feel confident deploying?

JavaCro 2026 · October 13, 2026

About Philip

- Software Engineer from Herzogenaurach (HQ of adidas & Puma), Germany 🍻
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🍃
- Founder of PragmaTech GmbH - **Enabling Developers to Frequently Deliver** Software with **More Confidence**



Herzogenaurach, Bavaria

Participate During the Talk

Scan the QR code to **anonymously** submit answers:



Why Test Software?



In 2026, most of our code is not written by us.



Frog told the LLM to write tests for all the code it wrote.

"There," he said. "Now the model can't write broken or buggy code."

"But the model can write broken or buggy tests," said Toad.

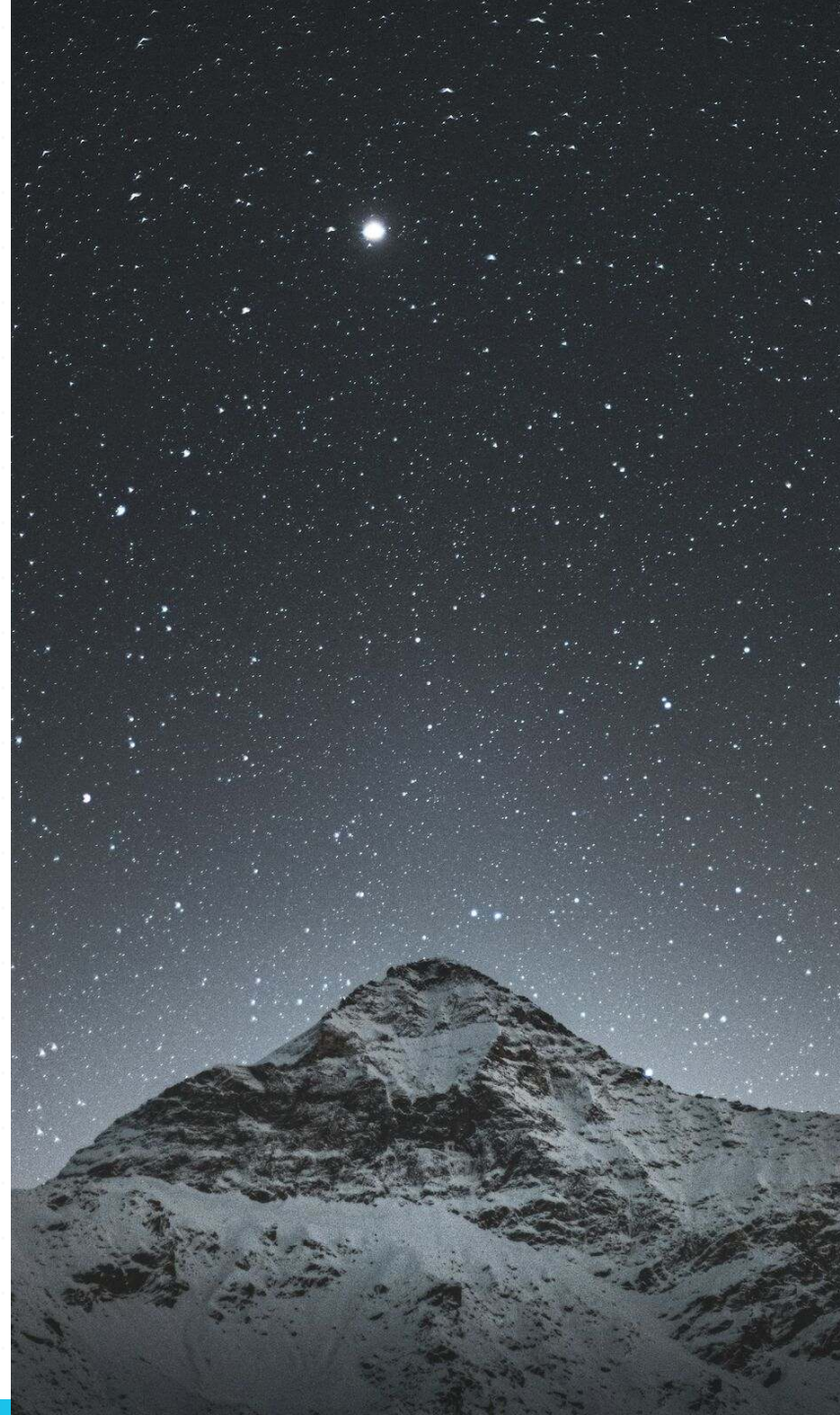
"That is true," said Frog.

My Overall Northstar for Engineering Excellence

Imagine seeing this pull request on a Friday afternoon:



How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?



Good tests don’t just catch bugs - they give you fast feedback and confident deployments.





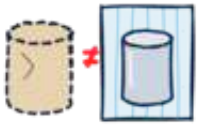
















Golden Hammer

Context Tax

Production Parity

Phantom Commit

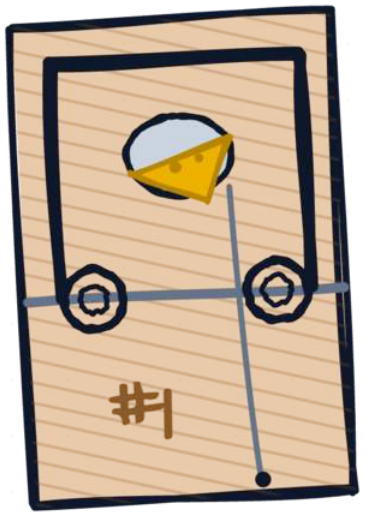
Green Lie



TESTING TRAP #1

The Golden Hammer

If all you have is a hammer, everything looks like a nail.



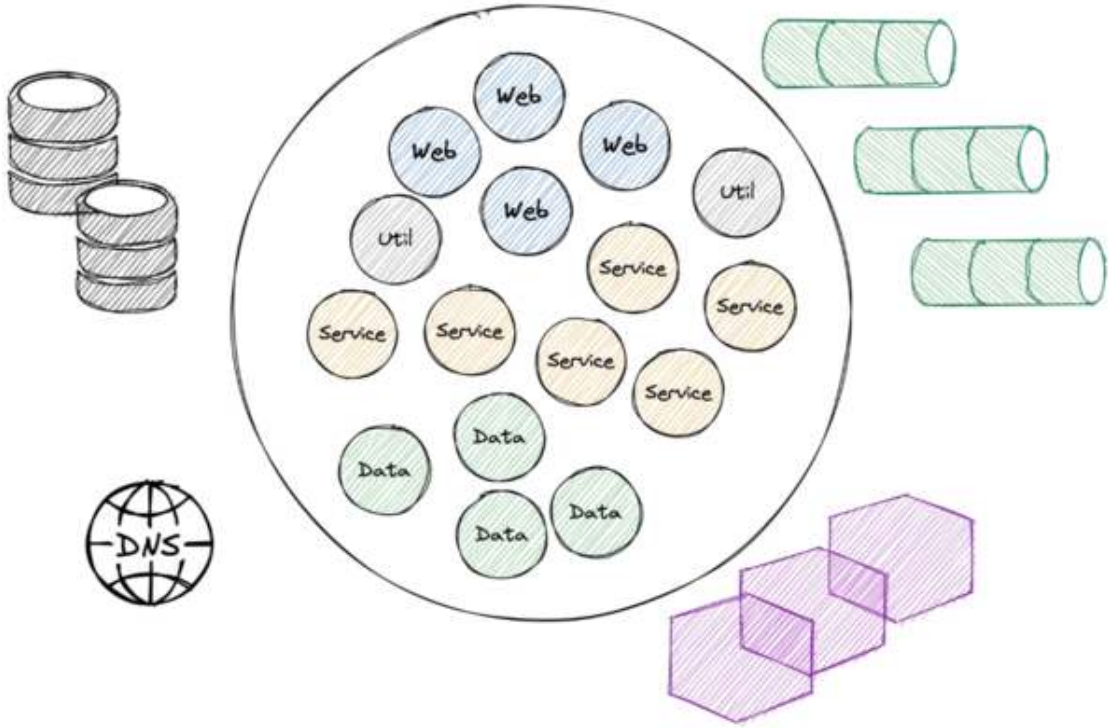


The `@SpringBootTest` Obsession

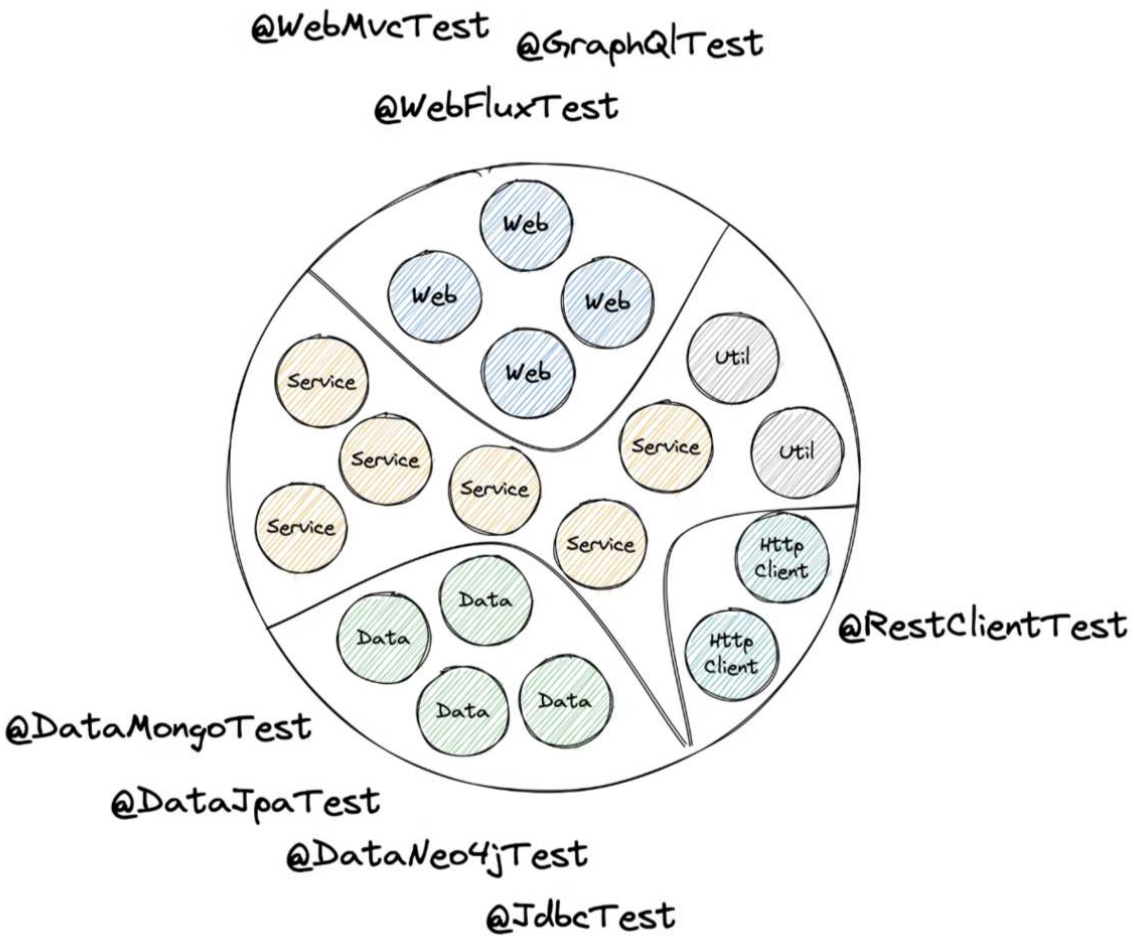
```
@SpringBootTest
class ApplicationTest {

  @Test
  void contextLoads() {
  }

}
```



You Don’t Always Need the Entire Context



Decision Paralysis: When to Include a Context

A simplified decision table:

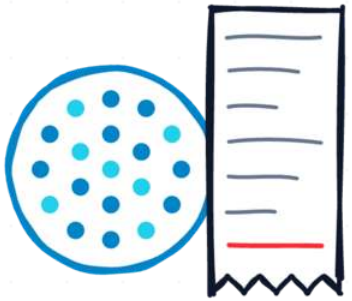
Question	Tool
Does my business logic work?	Plain JUnit + Mockito, no Spring
Does my HTTP layer map, validate, serialize?	`@WebMvcTest`
Does my query return what I think?	`@DataJpaTest`
Does my client talk to the remote API?	`@RestClientTest` (including a mock HTTP server)
Does the whole thing start and work end to end?	`@SpringBootTest`

TESTING TRAP #2

The Context Tax

Every `ApplicationContext` you use for testing, comes with a price.





The No. #1 Spring Test Hidden Gem

- Starting the `ApplicationContext` comes with a cost: test execution time
- Every context launch (sliced or full) takes multiple seconds
- Spring Test fixes this with: **TestContext Context Caching**

Results from one of our clients:



Context Caching in a Nutshell

```
1   // DefaultContextCache.java
2   private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LinkedHashMap<>(32, 0.75f, true));
```

- Spring test builds a unique `ApplicationContext` configuration, by determining which profiles, properties, classes, etc. are included in the test context (stored inside `MergedContextConfiguration`)
- If a subsequent test requires the exact same configuration, Spring simply hands over a "hot" context
- The goal is to reduce the context configuration variety to a bare minimum to have as many cache hits as possible -> fast test execution

How to Identify Context Restarts - Simplified

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] org.springframework.test.context.cache   : Spring test
 ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
 hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] tractDirtiesContextTestExecutionListener : After test class:
 context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
  = [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
 '{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
 propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
 .SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
 .filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
 .DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

How to Identify Context Restarts - Visualized



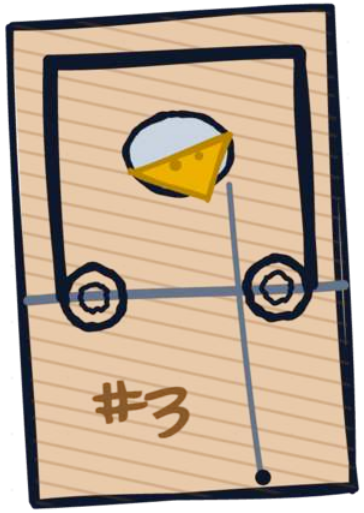
An open-source Spring Test utility that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

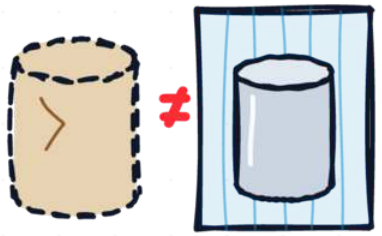
Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

TESTING TRAP #3

The Production Parity Trap

It works on my machine.





Where Tests Drift Away From Production

Every test environment takes shortcuts.

Each one is a small drift - and drifts compound:

- **Database**: in-memory H2 instead of the production engine and version
- **Servlet container**: `@SpringBootTest` defaults to a **mocked** servlet environment - `webEnvironment = RANDOM_PORT` starts the real one (Tomcat, etc.)
- **External services**: mocked mail, object storage, message queues, IDPs
- **Schema**: Hibernate `ddl-auto` in tests, Flyway/Liquibase migrations in production
- **Environment**: clock, timezone, locale, JVM flags, OS of the CI runner

The Classic Drift: The In-Memory Database

```
@DataJpaTest
class OrderRepositoryTest {

  @Autowired private OrderRepository orderRepository;

  @Test
  void shouldFindOverdueOrders() {
    // green in two seconds - on an H2 that pretends to be PostgreSQL
  }
}
```

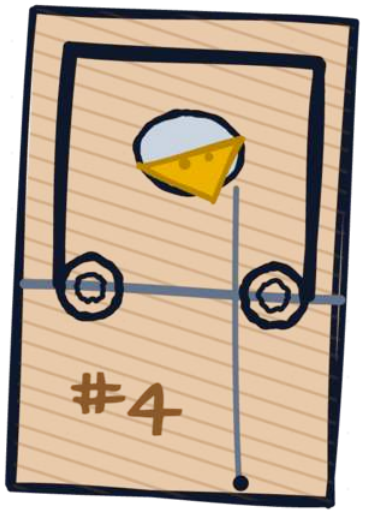
`@DataJpaTest` silently swaps in an embedded database (`replace = Replace.ANY` is the default). H2 in PostgreSQL mode is a **costume, not the engine**.

Disarm It: Close the Gap Step by Step

- **Understand your shortcuts**: know where your test environment drifts from production - and decide which drifts you can live with
- **Use Testcontainers**: run the real database, pinned to the production version (`postgres:17.2`) - `@ServiceConnection` wires it up; LocalStack, Kafka, Keycloak, and friends run as containers too
- **Test the app inside Docker**: start the container image you actually ship and run tests against it (JVM flags, base image, OS-level surprises)
- **Canary-test production continuously**: catch what no test environment can - expired SSL certs, rate limits, lost connectivity to cloud resources

TESTING TRAP #4

The Phantom Commit



Schrödingers database commit or `LazyInitializationException`.



The Green Test That Proves Nothing

```java
@Entity
public class Customer {

  @Id @GeneratedValue
  private Long id;
  private String email;

  public Customer(String email) { this.email = email; }

  // what's the catch here?
}
```

```java
@Test
void greenButMeaningless() {
  Long id = customerRepository.save(new Customer("duke@pragmatech.digital")).getId();
  assertThat(customerRepository.findById(id)).isPresent(); // green test, failing production
}
```

What Hibernate Does Behind Your Back

```
1   // as per JPA spec, entity classes require a default constructor
2   protected Customer() {
3
4   }
5
6   @Test
7   void greenButMeaningless() {
8     Long id = customerRepository.save(new Customer("duke@pragmatech.digital")).getId();
9     assertThat(customerRepository.findById(id)).isPresent(); // only interacts with first-level cache
10  }
```

- The entity is **never materialized from a real row**: the missing no-arg constructor stays invisible until the first production query throws `InstantiationException`
- **Persistence context (first-level cache)**: every entity you save or load lives here - `findById` returns the cached instance, **no SQL executed**
- Consider the `TestEntityManager.persistFlushFind()`

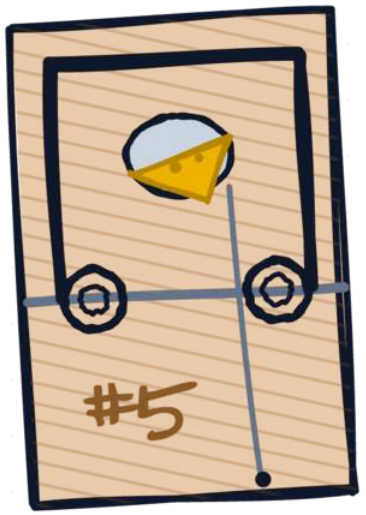
Be Careful on the Transaction Boundary

- `@Transactional` in a test **rolls back** the transaction : no commit-time constraint checks, no visible change
- `@TransactionalEventListener(phase = AFTER_COMMIT)` **may not fire** in the test
- Lazy loading works inside the test transaction and throws `LazyInitializationException` in production
- The gold standard: `@SpringBootTest(webEnvironment = RANDOM_PORT)` + a real HTTP call - the request runs in its **own transaction**, flushes, commits, and reads real rows

TESTING TRAP #5

The Green Lie

Never trust a test you haven't seen failing.





Be Aware of: Watermelon Tests

... green on the outside, red on the inside.

- **100% coverage** and still broken: coverage measures which lines *ran*, not which behavior was *verified*
- A test that asserts the mock works as previously instructed
- Auto-generated tests can give you the **feeling** of safety
- Agents produce these at scale: plausible names, green checks, zero judgment

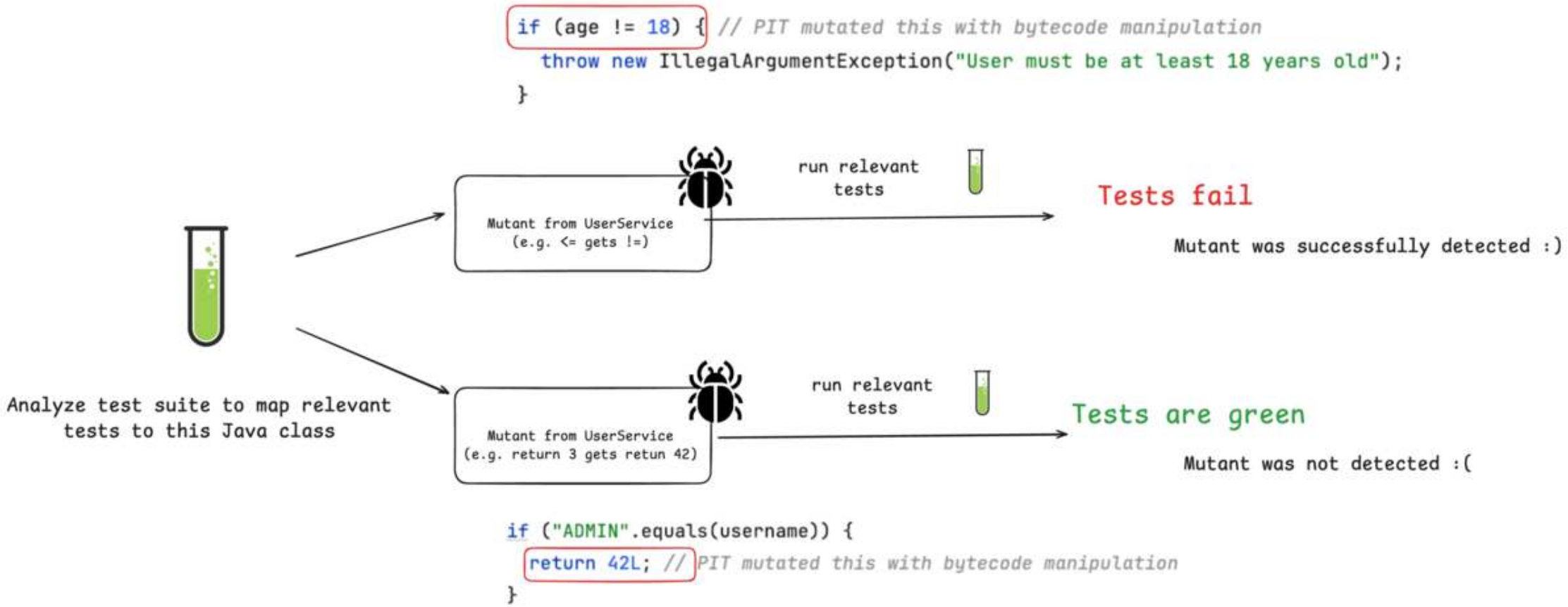
Let’s Challenge Code Coverage

Imagine a set of unit tests for this isolated business logic:

```
1   public Long registerUser(int age, String username) {
2
3     if (age <= 18) {
4       throw new IllegalArgumentException("User must be at least 18 years old");
5     }
6
7     if ("ADMIN".equalsIgnoreCase(username)) {
8       throw new IllegalArgumentException("Username 'ADMIN' is not allowed");
9     }
10
11    // ...
12
13  }
```

Idea: Introduce Regressions to Verify Test Quality

... with the help of PIT:







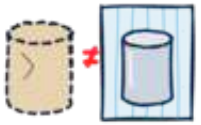
















Golden Hammer

✓ test the smallest slice

Context Tax

✓ reuse one cached context

Production Parity

✓ prod-like dependencies

Phantom Commit

✓ test the commit boundary

Green Lie

✓ mutation-test your tests



AI writes tests in seconds. Trusting them still takes human judgment.

What I Do to Build Confidence in the Agentic Coding Era

- **Review generated tests** like a pull request from a new hire
- **Put the bridges in the repo**: test slices, shared test configuration, Testcontainers best practices
- **Make the rules executable**: ArchUnit rules, my testing conventions in `CLAUDE.md` / `AGENTS.md`, mutation score thresholds (PIT) in CI
- **Custom skills**: Define once for each project how automated testing is tackled, best practices, antipatterns, etc.
- **Keep the feedback loop tight**: fast, trustworthy tests let the agent iterate without me watching every step (and reduce context switches)

My Agentic Testing Setup for Spring Boot

Define a **skillset** for fast & comprehensive tests, including a **test strategy** for the given project:

```
.claude/skills/
├── unit-testing              fast tests without context bloat
├── slice-testing             right-sized Spring context slices
├── slice-testing-webmvc      web layer with @WebMvcTest
├── integration-testing       full-context tests that stay fast
├── e2e-testing               user journeys against the running app
├── testcontainers-setup      real infrastructure, reused containers
└── test-setup-reviewer       flags test anti-patterns for you
```

Each skill includes rules, references, best-practices and antipatterns described as code and text.

You can delegate the typing.

You can't delegate the ownership.

You get paged at 3 AM.

Invest in a test suite that gives you confidence in every commit.



Top 5 Spring Boot Testing Mistakes Developers Trap Into

Joyful Testing!

Join my **Spring Boot Testing Newsletter** for best practices, recipes and quick wins in your inbox:
rieckpil.de/newsletter

